

RAPORT Z LISTY

Planowanie z wykorzystaniem języka PDDL

Sztuczna Inteligencja i Inżynieria Wiedzy — Lista 3

10.05.2026

Języki i narzędzia: PDDL | editor.planning.domains

Solvery: ENHSP, OPTIC, BFWS-ff, Metric-FF

Pliki: domain_zad1.pddl | problem_zad1.pddl | domain_zad2.pddl | problem_zad2.pddl |
domain_zad3.pddl | problem_zad3.pddl

Spis treści

1. Zadanie 1 — Transport paczek
2. Zadanie 2 — Robot odkurzaczy
3. Zadanie 3 — Robot z piłkami
4. Wnioski końcowe
5. Pliki PDDL — Zadanie 1
6. Pliki PDDL — Zadanie 2
7. Pliki PDDL — Zadanie 3

1. Zadanie 1 — Transport paczek (25 pkt)

1.1. Opis problemu

Zadanie polegało na zaprojektowaniu planu logistycznego dla systemu transportu trzech paczek między pięcioma lokalizacjami (Warszawa, Kraków, Gdańsk, Hamburg, Rotterdam) z użyciem trzech środków transportu: ciężarówek (drogi), samolotu (trasy lotnicze) oraz statku (trasy morskie). Każdy środek transportu ma przypisany koszt przejazdu między lokalizacjami, a celem optymalizacji jest minimalizacja sumy kosztów całego planu.

1.2. Model logiczny — rozszerzenia PDDL

W projekcie zastosowano następujące rozszerzenia języka PDDL:

- `:strips` — podstawowy model operacji STRIPS definiujący warunki wstępne i efekty akcji
- `:typing` — typowanie obiektów: rozróżnienie typów `location`, `vehicle` (z podtypami `truck`, `plane`, `ship`) oraz `package`
- `:negative-preconditions` — możliwość używania warunków negatywnych (`not ...`) w definicjach akcji
- `:numeric-fluents` — zmienne numeryczne do przechowywania kosztów tras (`road-dist`, `air-dist`, `sea-dist`) oraz akumulatora `total-cost`
- `:action-costs` — kosztowanie akcji z użyciem (`increase (total-cost) ...`) w efektach każdej akcji; cel planu zdefiniowany jako (`:metric minimize (total-cost)`)

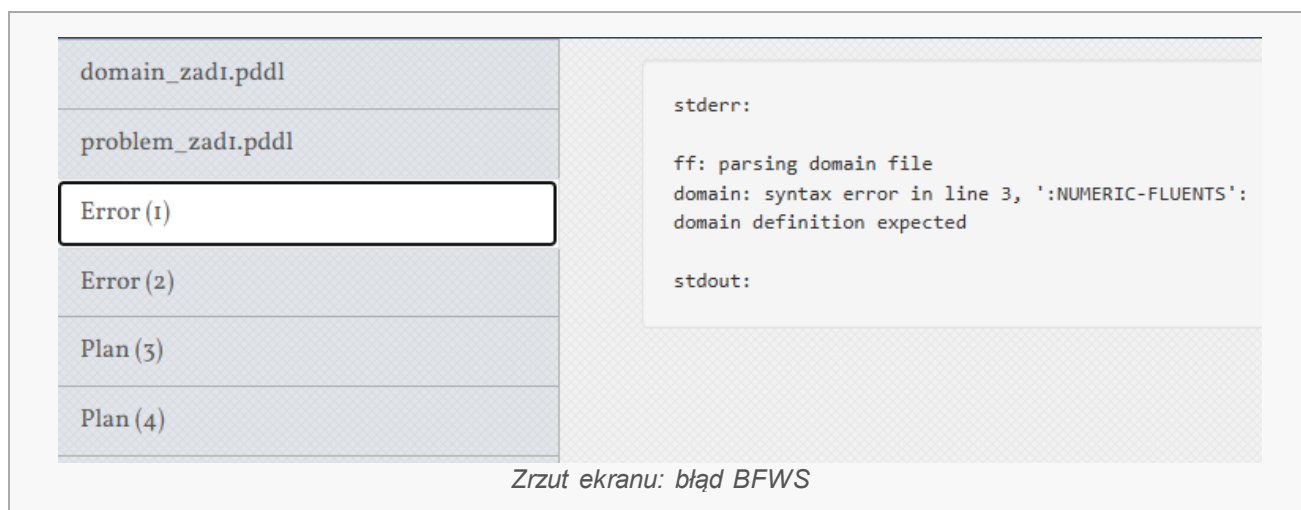
Topologia transportu obejmuje trzy warstwy:

- Sieć drogowa: Warszawa ↔ Kraków (koszt 3), Warszawa ↔ Gdańsk (koszt 6), Kraków ↔ Gdańsk (koszt 7)
- Sieć lotnicza: Warszawa ↔ Hamburg, Kraków ↔ Hamburg (koszt zmienny — eksperyment)
- Sieć morska: Gdańsk ↔ Rotterdam (koszt 8), Gdańsk ↔ Hamburg (koszt 4)

1.3. Napotkane problemy i błędy

1.3.1. Błąd solvera BFWS-ff — brak obsługi `:numeric-fluents`

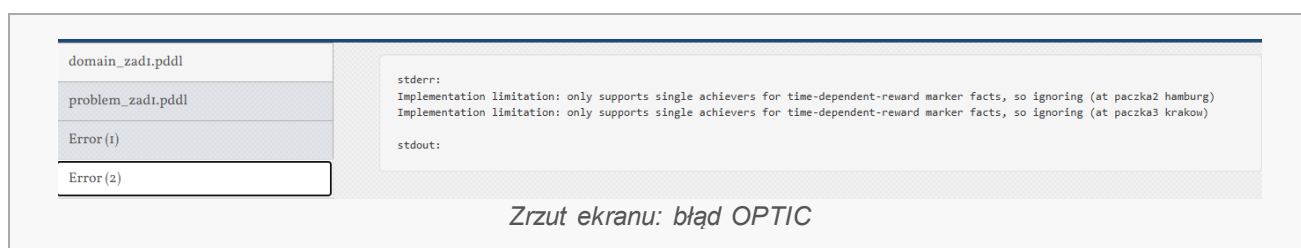
Pierwszą próbą uruchomienia planera był solver BFWS-ff (Best-First Width Search). Solver zwrócił błąd składni już na etapie parsowania pliku `domain.pddl`:



Błąd wynikał z faktu, że solver BFWS-ff nie obsługuje rozszerzenia :numeric-fluents.

1.3.2. Błąd solvera OPTIC — ograniczenia implementacyjne

Kolejną próbą był solver OPTIC. W przeciwieństwie do BFWS-ff, OPTIC zaakceptował pliki PDDL i nie zwrócił błędu parsowania. Jednak w stderr pojawiły się następujące ostrzeżenia:



Oznacza to, że OPTIC zignorował dwa z trzech warunków celu — cele dotyczące paczka2 i paczka3 zostały pominięte, a solver szukał planu wyłącznie dla paczka1. Wynika to z ograniczenia implementacyjnego OPTIC: solver nie potrafi obsłużyć wielu niezależnych celów osiąganych przez różne sekwencje akcji w kontekście fluents numerycznych. W efekcie wygenerowany plan byłby niekompletny i niepoprawny z punktu widzenia oryginalnego problemu. Z tego powodu do ostatecznych eksperymentów wybrano solver ENHSP, który obsługuje pełny zakres PDDL 2.1 i poprawnie traktuje wszystkie warunki celu.

1.4. Wyniki eksperymentów

1.4.1. Eksperyment 1 — koszt lotu = 2

Pierwszy eksperyment wykonany solverem ENHSP z oryginalnym kosztem trasy lotniczej równym 2 (air-dist = 2).

domain_zad1.pddl
problem_zad1.pddl
Error (1)
Error (2)
Plan (3)
Plan (4)
Error (5)
domain_zad2.pddl
problem_zad2.pddl
Plan (6)
domain_zad3.pddl
problem_zad3.pddl
Plan (7)
Plan (9)
Plan (10)
Plan (11)
Error (12)
Error (13)
Error (14)
Error (15)

Found Plan (output)

Raw Result

```

domain parsed
problem parsed
grounding..
grounding time: 66
aibr preprocessing
|f|:39
|x|:0
|a|:92
|p|:0
|e|:0
h1 setup time (msec): 12
g(n)= 2.0 h(n)=27.0
g(n)= 3.0 h(n)=26.0
g(n)= 9.0 h(n)=20.0
g(n)= 10.0 h(n)=19.0
g(n)= 11.0 h(n)=18.0
g(n)= 19.0 h(n)=10.0
g(n)= 20.0 h(n)=9.0
g(n)= 24.0 h(n)=8.0
g(n)= 27.0 h(n)=5.0
g(n)= 28.0 h(n)=4.0
g(n)= 31.0 h(n)=3.0
g(n)= 33.0 h(n)=1.0
problem solved

found plan:
0.0: (fly plane1 warszawa hamburg)
1.0: (load paczka1 truck1 warszawa)
2.0: (drive truck1 warszawa gdansk)
3.0: (unload paczka1 truck1 gdansk)
4.0: (load paczka1 ship1 gdansk)
5.0: (sail ship1 gdansk rotterdam)
6.0: (unload paczka1 ship1 rotterdam)
7.0: (drive truck2 krakow warszawa)
8.0: (load paczka3 truck2 warszawa)
9.0: (drive truck2 warszawa krakow)
10.0: (unload paczka3 truck2 krakow)
11.0: (fly plane1 hamburg krakow)
12.0: (load paczka2 plane1 krakow)
13.0: (fly plane1 krakow hamburg)
14.0: (unload paczka2 plane1 hamburg)

plan-length:15
metric (search):34.0
planning time (msec): 37
heuristic time (msec): 17
search time (msec): 32
expanded nodes:18
states evaluated:143
number of dead-ends detected:0
number of duplicates detected:26

```

Zrzut ekranu: wynik ENHSP — koszt lotu = 2, plan 15 kroków, metryka 34.0

Parametr	Wartość
Długość planu	15 kroków
Koszt metryczny	34.0
Czas planowania	37 ms
Expanded nodes	18
States evaluated	143
Dead-ends	0

Wygenerowany plan:

```
0.0: (fly plane1 warszawa hamburg)
1.0: (load paczka1 truck1 warszawa)
2.0: (drive truck1 warszawa gdansk)
3.0: (unload paczka1 truck1 gdansk)
4.0: (load paczka1 ship1 gdansk)
5.0: (sail ship1 gdansk rotterdam)
6.0: (unload paczka1 ship1 rotterdam)
7.0: (drive truck2 krakow warszawa)
8.0: (load paczka3 truck2 warszawa)
9.0: (drive truck2 warszawa krakow)
10.0: (unload paczka3 truck2 krakow)
11.0: (fly plane1 hamburg krakow)
12.0: (load paczka2 plane1 krakow)
13.0: (fly plane1 krakow hamburg)
14.0: (unload paczka2 plane1 hamburg)
```

Solver poprawnie wykorzystał wszystkie trzy środki transportu. Paczka1 trafiła do Rotterdam drogą morską przez Gdańsk; paczka2 doleciała do Hamburga samolotem; paczka3 wróciła ciężarówką do Krakowa. Wynik 0 dead-endów świadczy o sprawnym działaniu heurystyki ENHSP.

1.4.2. Eksperyment 2 — zmiana kosztu lotu na 20

W drugim eksperymencie koszt trasy lotniczej (air-dist) został zwiększony z 2 do 20, symulując sytuację, w której transport lotniczy jest znacznie droższy od alternatyw.

domain_zad1.pddl
problem_zad1.pddl
Error (1)
Error (2)
Plan (3)
Plan (4)
Error (5)
domain_zad2.pddl
problem_zad2.pddl
Plan (6)
domain_zad3.pddl
problem_zad3.pddl
Plan (7)
Plan (9)
Plan (10)
Plan (11)
Error (12)
Error (13)
Error (14)
Error (15)

Found Plan (output)

Raw Result

```

domain parsed
problem parsed
grounding..
grounding time: 59
aibr preprocessing
|f|:39
|x|:0
|a|:92
|p|:0
|e|:0
h1 setup time (msec): 11
g(n)= 1.0 h(n)=37.0
g(n)= 8.0 h(n)=30.0
g(n)= 9.0 h(n)=29.0
g(n)= 10.0 h(n)=28.0
g(n)= 16.0 h(n)=26.0
g(n)= 17.0 h(n)=25.0
g(n)= 24.0 h(n)=18.0
g(n)= 25.0 h(n)=17.0
g(n)= 26.0 h(n)=16.0
g(n)= 27.0 h(n)=15.0
g(n)= 28.0 h(n)=14.0
g(n)= 37.0 h(n)=13.0
g(n)= 45.0 h(n)=5.0
g(n)= 49.0 h(n)=1.0
problem solved

found plan:
0.0: (load paczka2 truck2 krakow)
1.0: (drive truck2 krakow gdansk)
2.0: (load paczka1 truck1 warszawa)
3.0: (load paczka3 truck1 warszawa)
4.0: (drive truck1 warszawa gdansk)
5.0: (unload paczka1 truck1 gdansk)
6.0: (drive truck1 gdansk krakow)
7.0: (unload paczka3 truck1 krakow)
8.0: (unload paczka2 truck2 gdansk)
9.0: (load paczka1 ship1 gdansk)
10.0: (load paczka2 ship1 gdansk)
11.0: (sail ship1 gdansk rotterdam)
12.0: (unload paczka1 ship1 rotterdam)
13.0: (sail ship1 rotterdam gdansk)
14.0: (sail ship1 gdansk hamburg)
15.0: (unload paczka2 ship1 hamburg)

plan-length:16
metric (search):50.0
planning time (msec): 39
heuristic time (msec): 22
search time (msec): 36
expanded nodes:17
states evaluated:147
number of dead-ends detected:0
number of duplicates detected:23

```

Zrzut ekranu: wynik ENHSP — koszt lotu = 20, plan 16 kroków, metryka 50.0

Parametr	Eksperyment 1 (lot = 2)	Eksperyment 2 (lot = 20)
Długość planu	15 kroków	16 kroków
Koszt metryczny	34.0	50.0
Samolot użyty?	Tak	Nie
Statek użyty?	Tak	Tak
Dead-ends	0	0

Wygenerowany plan (eksperyment 2):

```

0.0: (load paczka2 truck2 krakow)
1.0: (drive truck2 krakow gdansk)
2.0: (load paczka1 truck1 warszawa)
3.0: (load paczka3 truck1 warszawa)
4.0: (drive truck1 warszawa gdansk)
5.0: (unload paczka1 truck1 gdansk)
6.0: (drive truck1 gdansk krakow)
7.0: (unload paczka3 truck1 krakow)
8.0: (unload paczka2 truck2 gdansk)
9.0: (load paczka1 ship1 gdansk)
10.0: (load paczka2 ship1 gdansk)
11.0: (sail ship1 gdansk rotterdam)
12.0: (unload paczka1 ship1 rotterdam)
13.0: (sail ship1 rotterdam gdansk)
14.0: (sail ship1 gdansk hamburg)
15.0: (unload paczka2 ship1 hamburg)

```

Kluczowa obserwacja: po podniesieniu kosztu lotniczego do 20 solver całkowicie zrezygnował z samolotu. Paczka2 trafiła do Hamburga alternatywną trasą — ciężarówką do Gdańska, a następnie statkiem. Solver posłał nawet statek z powrotem z Rotterdamu do Gdańska, by zabrać paczka2, gdyż trasa morska (koszt $4+4+4 = 12$) w dalszym ciągu jest tańsza niż lot (koszt 20). To pokazuje, że model z :numeric-fluents poprawnie modeluje optymalizację kosztów w zależności od topologii sieci transportowej.

1.5. Wnioski

Zastosowanie rozszerzenia :numeric-fluents i :action-costs umożliwia realistyczne modelowanie kosztów transportu i ich wpływu na wybór trasy. ENHSP okazał się jedynym solverem dostępnym online obsługującym te rozszerzenia — BFWS-ff i OPTIC odrzuciły domenę z błędem parsowania. Zmiana jednego parametru kosztowego radykalnie zmienia strategię planera, co potwierdza poprawność modelu.

2. Zadanie 2 — Robot odkurzacz (15 pkt)**2.1. Opis problemu**

Zadanie polegało na zaprojektowaniu planu dla robota sprząającego, który ma odwiedzić i posprzątać trzy pokoje. Robot startuje w pokoj1; wszystkie pokoje są brudne. Topologia połączeń jest liniowa: pokoj1 ↔ pokoj2 ↔ pokoj3.

2.2. Model logiczny

Domena zawiera dwa typy obiektów: robot i room. Predykaty at, dirty, clean i connected modelują odpowiednio pozycję robota, stan czystości pokoi i strukturę połączeń. Akcja clean wymaga warunku (dirty ?p) i (not (clean ?p)), co zapobiega wielokrotnemu sprzątaniu tego

samego pokoju. Akcja move sprawdza (connected ?from ?to), ograniczając możliwe przejścia do faktycznie połączonych pokoi.

Rozszerzenia użyte w domenie: :strips (model STRIPS), :typing (typowanie robot/room), :negative-preconditions (warunek (not (clean ?p)) w akcji clean).

2.3. Wynik planera — ENHSP

domain_zad1.pddl

problem_zad1.pddl

Error (1)

Error (2)

Plan (3)

Plan (4)

Error (5)

domain_zad2.pddl

problem_zad2.pddl

Plan (6)

domain_zad3.pddl

problem_zad3.pddl

Plan (7)

Plan (9)

Plan (10)

Plan (11)

Found Plan (output)

Raw Result

```

domain parsed
problem parsed
grounding..
grounding time: 22
aibr preprocessing
|f|:9
|x|:0
|a|:7
|p|:0
|e|:0
h1 setup time (msec): 6
g(n)= 1.0 h(n)=5.0
g(n)= 2.0 h(n)=3.0
g(n)= 3.0 h(n)=2.0
g(n)= 4.0 h(n)=1.0
problem solved

found plan:
0.0: (clean robot1 pokoj1)
1.0: (move robot1 pokoj1 pokoj2)
2.0: (clean robot1 pokoj2)
3.0: (move robot1 pokoj2 pokoj3)
4.0: (clean robot1 pokoj3)

plan-length:5
metric (search):5.0
planning time (msec): 23
heuristic time (msec): 1
search time (msec): 20
expanded nodes:6
states evaluated:9
number of dead-ends detected:0
number of duplicates detected:2

```

Zrzut ekranu: wynik ENHSP — zadanie 2, plan 5 kroków, metryka 5.0

Parametr	Wartość
Długość planu	5 kroków
Koszt metryczny	5.0
Czas planowania	23 ms
Expanded nodes	6
States evaluated	9
Dead-ends	0

Wygenerowany plan:

```
0.0: (clean robot1 pokoj1)
1.0: (move robot1 pokoj1 pokoj2)
2.0: (clean robot1 pokoj2)
3.0: (move robot1 pokoj2 pokoj3)
4.0: (clean robot1 pokoj3)
```

2.4. Analiza

Plan jest optymalny — 5 kroków to minimum dla liniowej topologii połączeń przy starcie w pokoj1. Robot wykonuje operację sprzątania i przejścia na zmianę, przemieszczając się jedynie naprzód. Mała liczba stanów (9 evaluated) i węzłów (6 expanded) potwierdza prostotę problemu. Brak dead-endów wskazuje na skuteczność heurystyki dla tego modelu.

Gdyby robot startował w pokoj2, plan miałby tę samą długość (5 kroków), lecz kolejność byłaby inna. Dodanie pokoju izolowanego (połączonego tylko z jednym pokojem) wymusiłoby powrót robota i wydłużyłoby plan.

3. Zadanie 3 — Robot z piłkami (10 pkt)

3.1. Opis problemu

Robot wyposażony w dwa ramiona (arm1, arm2) musi przenieść cztery piłki (ball1–ball4) z room1 do room2. Na początku robot i wszystkie piłki znajdują się w room1; oba ramiona są puste. Robot może chwycić piłkę wolnym ramieniem, przemieścić się między pokojami oraz odłożyć piłkę w bieżącym pokoju.

3.2. Model logiczny

Domena definiuje cztery typy: robot, room, ball, arm. Predykat arm-empty gwarantuje, że każde ramię trzyma co najwyżej jedną piłkę. Akcja pick-up wymaga (arm-empty ?a) i zdejmuje ten predykat z efektów, uniemożliwiając wzięcie więcej niż jednej piłki tym samym ramieniem. Model nie wymaga rozszerzeń numerycznych — wystarczą :strips i :typing.

3.3. Wynik planera — ENHSP (plan nieoptymalny)

domain_zad1.pddl
problem_zad1.pddl
Error (1)
Error (2)
Plan (3)
Plan (4)
Error (5)
domain_zad2.pddl
problem_zad2.pddl
Plan (6)
domain_zad3.pddl
problem_zad3.pddl
Plan (7)
Plan (9)
Plan (10)
Plan (11)
Error (12)
Error (13)
Error (14)
Error (15)

Found Plan (output)

Raw Result

```

domain parsed
problem parsed
grounding..
grounding time: 24
aibr preprocessing
|f|:20
|x|:0
|a|:36
|p|:0
|e|:0
h1 setup time (msec): 8
g(n)= 1.0 h(n)=11.0
g(n)= 2.0 h(n)=10.0
g(n)= 3.0 h(n)=9.0
g(n)= 5.0 h(n)=8.0
g(n)= 6.0 h(n)=7.0
g(n)= 7.0 h(n)=6.0
g(n)= 9.0 h(n)=5.0
g(n)= 10.0 h(n)=4.0
g(n)= 11.0 h(n)=3.0
g(n)= 13.0 h(n)=2.0
g(n)= 14.0 h(n)=1.0
problem solved

found plan:
0.0: (pick-up robot arm1 ball3 room1)
1.0: (move robot room1 room2)
2.0: (put-down robot arm1 ball13 room2)
3.0: (move robot room2 room1)
4.0: (pick-up robot arm1 ball1 room1)
5.0: (move robot room1 room2)
6.0: (put-down robot arm1 ball1 room2)
7.0: (move robot room2 room1)
8.0: (pick-up robot arm2 ball14 room1)
9.0: (move robot room1 room2)
10.0: (put-down robot arm2 ball14 room2)
11.0: (move robot room2 room1)
12.0: (pick-up robot arm2 ball12 room1)
13.0: (move robot room1 room2)
14.0: (put-down robot arm2 ball12 room2)

plan-length:15
metric (search):15.0
planning time (msec): 36
heuristic time (msec): 7
search time (msec): 30
expanded nodes:16
states evaluated:54
number of dead-ends detected:15
number of duplicates detected:14

```

Zrzut ekranu: wynik ENHSP — zadanie 3, plan 15 kroków (nieoptymalny)

Parametr	Wartość
Długość planu	15 kroków
Koszt metryczny	15.0
Czas planowania	36 ms
Expanded nodes	16
States evaluated	54
Dead-ends	15

Wygenerowany plan (ENHSP):

```
0.0: (pick-up robot arm1 ball3 room1)
1.0: (move robot room1 room2)
2.0: (put-down robot arm1 ball3 room2)
3.0: (move robot room2 room1)
4.0: (pick-up robot arm1 ball1 room1)
5.0: (move robot room1 room2)
6.0: (put-down robot arm1 ball1 room2)
7.0: (move robot room2 room1)
8.0: (pick-up robot arm2 ball4 room1)
9.0: (move robot room1 room2)
10.0: (put-down robot arm2 ball4 room2)
11.0: (move robot room2 room1)
12.0: (pick-up robot arm2 ball2 room1)
13.0: (move robot room1 room2)
14.0: (put-down robot arm2 ball2 room2)
```

Uwaga: ENHSP zastosował algorytm zachłanny (hill-climbing), który znalazł pierwsze dopuszczalne rozwiązanie. Robot przenosi piłki po jednej, ignorując fakt posiadania wolnego drugiego ramienia. Skutkuje to czterema kursami zamiast optymalnych dwóch. Obecność 15 dead-endów (brak w zadaniu 1 i 2) wskazuje, że przestrzeń stanów jest bardziej złożona, a heurystyka nie prowadzi bezpośrednio do optymalnego rozwiązania.

3.4. Wynik planera — OPTIC (plan optymalny)

Plan (10)
 Plan (11)
 Error (12)
 Error (13)
 Error (14)
 Error (15)

```

number of literals: 20
constructing lookup tables: [10%] [20%] [30%] [40%] [50%] [60%] [70%] [80%] [90%] [100%] [110%] [120%]
post filtering unreachable actions: [10%] [20%] [30%] [40%] [50%] [60%] [70%] [80%] [90%] [100%] [110%] [120%]
no semaphore facts found, returning
[8]:34no analytic limits found, not considering limit effects of goal-only operators[00m
Initial heuristic = 9.000, admissible cost estimate 0.000
b (8.000 | 0.000)b (7.000 | 0.001)b (6.000 | 0.002)b (5.000 | 0.002)b (4.000 | 0.004)b (3.000 | 0.004)b (2.000 | 0.005)b (1.000 | 0.005)(g)
; no metric specified - using makespan

; plan found with metric 0.006
; states evaluated so far: 15
; states pruned based on pre-heuristic cost lower bound: 0
; time 0.05
0.000: (pick-up robot arm1 ball14 room1) [0.001]
0.000: (pick-up robot arm2 ball13 room1) [0.001]
0.001: (move robot room1 room2) [0.001]
0.002: (put-down robot arm1 ball14 room2) [0.001]
0.002: (put-down robot arm2 ball13 room2) [0.001]
0.003: (move robot room2 room1) [0.001]
0.004: (pick-up robot arm1 ball12 room1) [0.001]
0.004: (pick-up robot arm2 ball11 room1) [0.001]
0.005: (move robot room1 room2) [0.001]
0.006: (put-down robot arm1 ball12 room2) [0.001]
0.006: (put-down robot arm2 ball11 room2) [0.001]

* all goal deadlines now no later than 0.006

resorting to best-first search
running wa* with w = 5.000, not restarting with goal states
b (8.000 | 0.000)b (7.000 | 0.001)b (6.000 | 0.002)b (5.000 | 0.002)b (4.000 | 0.004)b (3.000 | 0.004)b (2.000 | 0.005)b (1.000 | 0.005)
problem unsolvable
;;; solution found
; states evaluated: 108590
; cost: 0.006
; time 21.15
0.000: (pick-up robot arm1 ball14 room1) [0.001]
0.000: (pick-up robot arm2 ball13 room1) [0.001]
0.001: (move robot room1 room2) [0.001]
0.002: (put-down robot arm1 ball14 room2) [0.001]
0.002: (put-down robot arm2 ball13 room2) [0.001]
0.003: (move robot room2 room1) [0.001]
0.004: (pick-up robot arm1 ball12 room1) [0.001]
0.004: (pick-up robot arm2 ball11 room1) [0.001]
0.005: (move robot room1 room2) [0.001]
0.006: (put-down robot arm1 ball12 room2) [0.001]
0.006: (put-down robot arm2 ball11 room2) [0.001]

```

Zrzut ekranu: wynik OPTIC — zadanie 3, plan 11 kroków (optymalny, oba ramiona jednocześnie)

Parametr	ENHSP (nieoptymalny)	OPTIC (optymalny)
Długość planu	15 kroków	11 kroków
Liczba kursów (move)	8	4
Ramiona używane jednocześnie	Nie	Tak
States evaluated	54	15 (pierwsza faza)
Dead-ends	15	0

Wygenerowany plan (OPTIC):

```

0.000: (pick-up robot arm1 ball14 room1)
0.000: (pick-up robot arm2 ball13 room1)
0.001: (move robot room1 room2)
0.002: (put-down robot arm1 ball14 room2)
0.002: (put-down robot arm2 ball13 room2)
0.003: (move robot room2 room1)
0.004: (pick-up robot arm1 ball12 room1)
0.004: (pick-up robot arm2 ball11 room1)

```

```
0.005: (move robot room1 room2)
0.006: (put-down robot arm1 ball2 room2)
0.006: (put-down robot arm2 ball1 room2)
```

OPTIC zastosował model temporalny — równoczesne akcje (timestamp 0.000 dla obu pick-up) możliwe są dzięki temu, że oba ramiona są niezależnymi zasobami. Solver traktuje równoczesne podniesienie obu piłek jako akcje niekolidujące. Wynikiem jest plan 11-krokowy z tylko dwoma kursami, co jest optymalnym rozwiązaniem dla czterech piłek i dwóch ramion.

3.5. Analiza i wnioski

Pliki PDDL były poprawne od początku — różnica w wynikach wynika wyłącznie z przyjętej przez solver strategii przeszukiwania. ENHSP (algorytm zachłanny) zadowolił się pierwszym dopuszczalnym rozwiązaniem, pomijając możliwość jednoczesnego użycia obu ramion. OPTIC (optymalizacja makespan) znalazł plan optymalny, rozpoznając niezależność ramion jako możliwość równoległości. Wynik pokazuje, że model PDDL z :strips i :typing jest wystarczający do poprawnego opisu problemu, jednak jakość planu zależy od doboru solvera i jego strategii przeszukiwania.

4. Wnioski końcowe

Wykonanie trzech zadań z listy nr 3 pozwoliło na zapoznanie się z językiem PDDL i jego praktycznym zastosowaniem w modelowaniu problemów planowania. Poniżej zestawiono kluczowe obserwacje:

- Wybór solvera ma kluczowe znaczenie. Solvery BFWS-ff i OPTIC dostępne online nie obsługują rozszerzenia :numeric-fluents i odrzucały domenę z błędem parsowania. Jedynym solverem zdolnym do obsługi pełnego modelu z kosztami był ENHSP.
- Rozszerzenie :numeric-fluents i :action-costs umożliwia realistyczne modelowanie kosztów i optymalizację trasy. Zmiana jednego parametru (koszt lotu z 2 na 20) spowodowała całkowitą zmianę strategii transportowej — solver zrezygnował z samolotu na rzecz drogi morskiej.
- Heurystyki zachłanne mogą generować plany suboptymalnych. W zadaniu 3 ENHSP znalazł plan 15-krokowy, pomijając możliwość przenoszenia dwóch piłek jednocześnie. Dopiero OPTIC z optymalizacją makespan wygenerował optymalny plan 11-krokowy.
- Model PDDL jest poprawny niezależnie od solvera. Różne wyniki w zadaniu 3 wynikały ze strategii przeszukiwania, a nie z błędów w plikach PDDL.
- Topologia sieci wpływa na długość i koszt planu. W zadaniu 1 złożona topologia (trzy warstwy transportu) daje solverowi elastyczność w wyborze trasy, co prowadzi do planów optymalnych kosztowo, choć niekoniecznie najkrótszych krokowo.

5. Pliki PDDL — Zadanie 1

5.1. domain_zad1.pddl

```
(define (domain transport)
  (:requirements :strips :typing :negative-preconditions
    :numeric-fluents :action-costs)

  (:types
    location vehicle package - object
    truck plane ship - vehicle
  )

  (:predicates
    (at ?obj - object ?loc - location)
    (in ?pkg - package ?veh - vehicle)
    (road-connected ?l1 - location ?l2 - location)
    (air-connected ?l1 - location ?l2 - location)
    (sea-connected ?l1 - location ?l2 - location)
    (can-road ?v - vehicle)
    (can-air ?v - vehicle)
    (can-sea ?v - vehicle)
  )

  (:functions
    (total-cost)
    (road-dist ?l1 - location ?l2 - location)
    (air-dist ?l1 - location ?l2 - location)
    (sea-dist ?l1 - location ?l2 - location)
  )

  (:action load
    :parameters (?pkg - package ?veh - vehicle ?loc - location)
    :precondition (and (at ?pkg ?loc) (at ?veh ?loc))
    :effect (and (in ?pkg ?veh) (not (at ?pkg ?loc))
      (increase (total-cost) 1))
  )

  (:action unload
    :parameters (?pkg - package ?veh - vehicle ?loc - location)
    :precondition (and (in ?pkg ?veh) (at ?veh ?loc))
    :effect (and (at ?pkg ?loc) (not (in ?pkg ?veh))
      (increase (total-cost) 1))
  )

  (:action drive
    :parameters (?t - truck ?from - location ?to - location)
    :precondition (and (at ?t ?from) (can-road ?t) (road-connected ?from ?to))
```

```

    :effect (and (at ?t ?to) (not (at ?t ?from))
                 (increase (total-cost) (road-dist ?from ?to)))
  )

  (:action fly
    :parameters (?p - plane ?from - location ?to - location)
    :precondition (and (at ?p ?from) (can-air ?p) (air-connected ?from ?to))
    :effect (and (at ?p ?to) (not (at ?p ?from))
                 (increase (total-cost) (air-dist ?from ?to)))
  )

  (:action sail
    :parameters (?s - ship ?from - location ?to - location)
    :precondition (and (at ?s ?from) (can-sea ?s) (sea-connected ?from ?to))
    :effect (and (at ?s ?to) (not (at ?s ?from))
                 (increase (total-cost) (sea-dist ?from ?to)))
  )
)

```

5.2. problem_zad1.pddl

```

(define (problem transport-p1)
  (:domain transport)

  (:objects
    warszawa krakow gdansk hamburg rotterdam - location
    truck1 truck2 - truck
    plane1      - plane
    ship1       - ship
    paczka1 paczka2 paczka3 - package
  )

  (:init
    (at truck1 warszawa) (at truck2 krakow)
    (at plane1 warszawa) (at ship1 gdansk)
    (at paczka1 warszawa) (at paczka2 krakow) (at paczka3 warszawa)
    (can-road truck1) (can-road truck2)
    (can-air plane1) (can-sea ship1)
    ;; Połączenia drogowe
    (road-connected warszawa krakow) (road-connected krakow warszawa)
    (road-connected warszawa gdansk) (road-connected gdansk warszawa)
    (road-connected krakow gdansk) (road-connected gdansk krakow)
    ;; Połączenia lotnicze
    (air-connected warszawa hamburg) (air-connected hamburg warszawa)
    (air-connected krakow hamburg) (air-connected hamburg krakow)
    ;; Połączenia morskie
    (sea-connected gdansk rotterdam) (sea-connected rotterdam gdansk)
  )
)

```

```
(sea-connected gdansk hamburg) (sea-connected hamburg gdansk)
;; Koszty drogowe
(= (road-dist warszawa krakow) 3) (= (road-dist krakow warszawa) 3)
(= (road-dist warszawa gdansk) 6) (= (road-dist gdansk warszawa) 6)
(= (road-dist krakow gdansk) 7)   (= (road-dist gdansk krakow) 7)
;; Koszty lotnicze (wersja 1: 2, wersja 2: 20)
(= (air-dist warszawa hamburg) 20) (= (air-dist hamburg warszawa) 20)
(= (air-dist krakow hamburg) 20)   (= (air-dist hamburg krakow) 20)
;; Koszty morskie
(= (sea-dist gdansk rotterdam) 8) (= (sea-dist rotterdam gdansk) 8)
(= (sea-dist gdansk hamburg) 4)   (= (sea-dist hamburg gdansk) 4)
(= (total-cost) 0)
)

(:goal (and
  (at paczka1 rotterdam)
  (at paczka2 hamburg)
  (at paczka3 krakow)
))

(:metric minimize (total-cost))
)
```

6. Pliki PDDL — Zadanie 2

6.1. domain_zad2.pddl

```
(define (domain robot-vacuum)
  (:requirements :strips :typing :negative-preconditions)

  (:types robot room - object)

  (:predicates
    (at ?r - robot ?p - room)
    (dirty ?p - room)
    (clean ?p - room)
    (connected ?p1 - room ?p2 - room)
  )

  (:action move
    :parameters (?r - robot ?from - room ?to - room)
    :precondition (and (at ?r ?from) (connected ?from ?to))
    :effect (and (at ?r ?to) (not (at ?r ?from)))
  )

  (:action clean
```



```
:parameters (?r - robot ?p - room)
:precondition (and (at ?r ?p) (dirty ?p) (not (clean ?p)))
:effect (and (clean ?p) (not (dirty ?p)))
)
)
```

6.2. problem_zad2.pddl

```
(define (problem robot-vacuum-p1)
  (:domain robot-vacuum)

  (:objects
    robot1 - robot
    pokoj1 pokoj2 pokoj3 - room
  )

  (:init
    (at robot1 pokoj1)
    (dirty pokoj1) (dirty pokoj2) (dirty pokoj3)
    (connected pokoj1 pokoj2) (connected pokoj2 pokoj1)
    (connected pokoj2 pokoj3) (connected pokoj3 pokoj2)
  )

  (:goal (and
    (clean pokoj1) (clean pokoj2) (clean pokoj3)
  ))
)
```

7. Pliki PDDL — Zadanie 3

7.1. domain_zad3.pddl

```
(define (domain ball-moving-robot)
  (:requirements :strips :typing)
  (:types robot room ball arm)

  (:predicates
    (at ?r - robot ?rm - room)
    (inroom ?b - ball ?rm - room)
    (holding ?a - arm ?b - ball)
    (arm-empty ?a - arm)
  )

  (:action move
    :parameters (?r - robot ?from - room ?to - room)
    :precondition (at ?r ?from)
```

```
    :effect (and (not (at ?r ?from)) (at ?r ?to))
  )

  (:action pick-up
    :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
    :precondition (and (at ?r ?rm) (inroom ?b ?rm) (arm-empty ?a))
    :effect (and (holding ?a ?b) (not (arm-empty ?a)) (not (inroom ?b ?rm)))
  )

  (:action put-down
    :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
    :precondition (and (at ?r ?rm) (holding ?a ?b))
    :effect (and (inroom ?b ?rm) (arm-empty ?a) (not (holding ?a ?b)))
  )
)
```

7.2. problem_zad3.pddl

```
(define (problem move-balls)
  (:domain ball-moving-robot)

  (:objects
    room1 room2 - room
    robot - robot
    ball1 ball2 ball3 ball4 - ball
    arm1 arm2 - arm
  )

  (:init
    (at robot room1)
    (inroom ball1 room1) (inroom ball2 room1)
    (inroom ball3 room1) (inroom ball4 room1)
    (arm-empty arm1) (arm-empty arm2)
  )

  (:goal (and
    (inroom ball1 room2) (inroom ball2 room2)
    (inroom ball3 room2) (inroom ball4 room2)
  ))
)
```